# Exp 10: Applying Performance Optimization Techniques (Indexing, Subforms, Query Tuning) to a Zoho Creator Application

## 1. Aim

To apply performance optimization techniques to a Zoho Creator application by structuring the form into sections, loading only the columns that a report needs, and tuning a Deluge query so that the database filters the records and the script counts the matches, then testing the changes with sample data.

## 2. Procedure

### 2.1 Optimization Requirements

An application slows down when forms are crowded, reports load more data than the user reads, and scripts fetch every record before filtering it. Three techniques were applied to counter this, as listed in Table 1.

| Technique | Component | Details |
|---|---|---|
| Organise the data model | Form sections (and subforms for repeated data) | Gym member form grouped into Member Info, Personal Details and Membership |
| Load less data | Report with selected columns | Member_Name Report shows four fields instead of the whole form |
| Tune the query | Deluge fetch with criteria and count() | Records are filtered by Preferred_Course on the database side, and only the count is used |
| Check the effect | Deluge info statement | The number of matching records is written to the execution log |

*Table 1: Optimization techniques and the components used*

### 2.2 Structuring the Form

The gym member form was arranged into three sections in the form builder (Figure 1), so that related fields sit together and the form loads and reads in logical blocks. The sections and their fields are listed in Table 2. Mandatory fields are marked with a red star, and the form contains only the fields that the business needs.

| Section | Fields |
|---|---|
| Member Info | Member_Name, Age, Membership_Type |
| Personal Details | Email_Address, Contact_Number, Residential_Address, Emergency_Contact_Name |
| Membership | Membership_Plan, Preferred_Activities, Fitness_Goals_Medical_Notes |

*Table 2: Sections and fields of the gym member form*

### 2.3 Indexing and Subforms

Creator does not provide a setting to create database indexes by hand, because the platform manages the storage itself. The practical way to get the benefit of an index is to search and filter on a field that holds a short, well-defined value, such as Membership_Type or Preferred_Course, instead of free text. The same idea applies to the design of the data model. A subform keeps repeated entries, such as the classes booked by one member, inside one record instead of in many loose records, whereas a lookup field links to a record in another form instead of copying its values into each row. Both reduce duplicated data, so there is less to store, fetch and keep consistent.

### 2.4 Loading Only the Needed Columns in a Report

The Member_Name Report was set up to show only the four fields that the front desk uses: Member_Name, Membership_Type, Email_Address and Contact_Number (Figure 2). The other fields of the form, such as the address and the fitness notes, are left out of the list view and are opened only when a single record is viewed. A

report that loads fewer columns and fewer rows opens faster, particularly as the number of members grows. The report holds six records, of which three are New Member, two are Renewal and one is Corporate Enrollment.

## 2.5 Tuning the Query with Deluge

Inside the workflow of the Admission Enquiry Form (Created > Successful form submission), a short Deluge script was written to count the records that have the same Preferred_Course as the new submission (Figure 3). The criteria are placed inside the fetch, so the database returns only the matching records, and the script then uses count() to report how many there are (Script 1).

```
matching_records = AddAdmission_Enquiry_Form[Preferred_Course == input.Preferred_Course];
info "Matching examples: " + matching_records.count();
```

*Script 1: Criteria-based fetch followed by a count of the matching records*

A slower way to get the same answer is to fetch all records and then test each one inside a loop, which makes the script read the whole form on every submission. The criteria-based fetch avoids this, and calling **count()** on the result gives the number without going through the records one by one. The editor also lists the field names of the form with their types while typing, as seen in Figure 3, which helps to avoid mistyped field names.

## 2.6 Testing the Optimizations

The changes were tested by submitting sample records and viewing the report and the log. Table 3 lists the cases covered and the behaviour that is expected.

| Test case | Action | Expected behaviour |
|---|---|---|
| Form layout | Open the gym member form in the builder | Fields appear under the three sections |
| Report columns | Open the Member_Name Report | Only the four selected columns are listed for each record |
| Matching course | Submit an enquiry with a course that was used before | Log shows “Matching examples: ” with the number of earlier records of that course |
| New course | Submit an enquiry with a course not used before | Log shows “Matching examples: 0” |

*Table 3: Test cases for the optimization changes*

# 3. Output

The following figures show the structured form, the report with selected columns, and the tuned Deluge script.



*Figure 1: Gym member form arranged into Member Info, Personal Details and Membership sections*

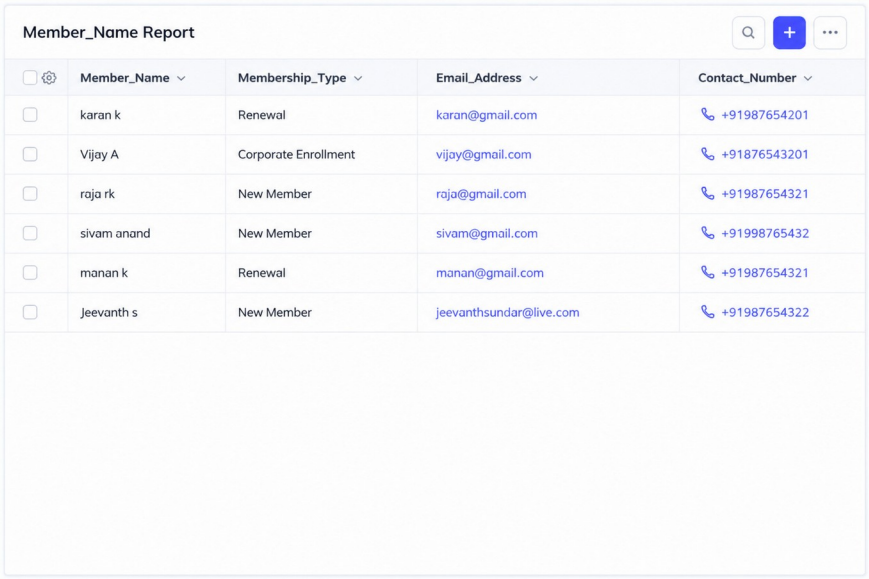



*Figure 2: Member_Name Report showing only Member_Name, Membership_Type, Email_Address and Contact_Number*

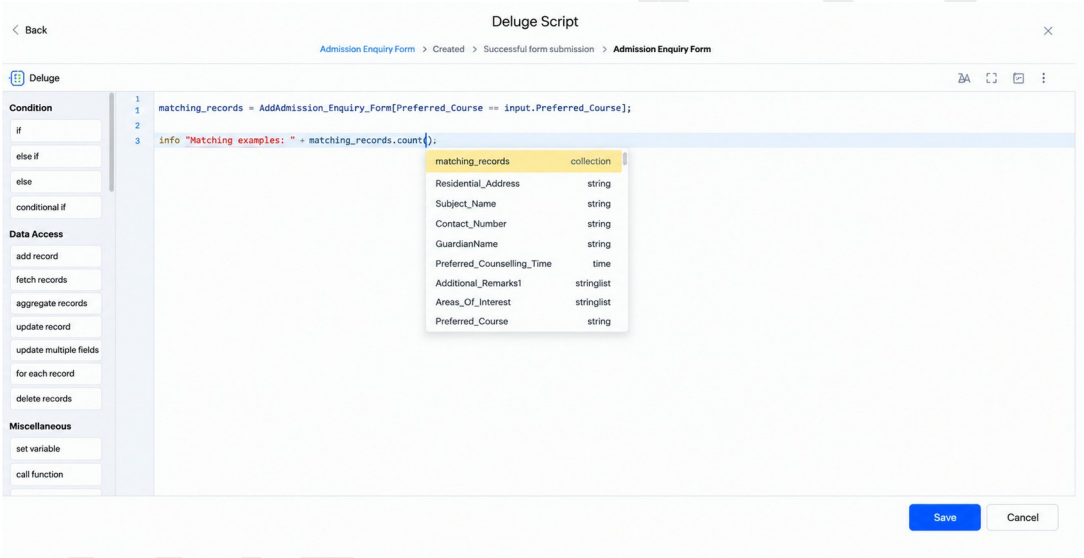



*Figure 3: Deluge script with a criteria-based fetch and count() in the Admission Enquiry Form workflow*

## Observations from the output

- Grouping the fields into sections keeps the form readable and makes it easy to find and change a field without opening unrelated ones.
- The report loads four columns instead of the whole record, so it stays quick to open as the number of members grows.
- Putting the criteria inside the fetch lets the database do the filtering, so the script does not read every record on each submission.
- Using count() and an info statement gives the answer and a visible check in the log with a single line of code.

# 4. Result

Performance optimization techniques were applied to the Zoho Creator application. The form was organised into sections, the report was limited to the columns that are needed, and a Deluge query was tuned with database-side criteria and a count. This completes the performance optimization requirements of this experiment.